Introduction

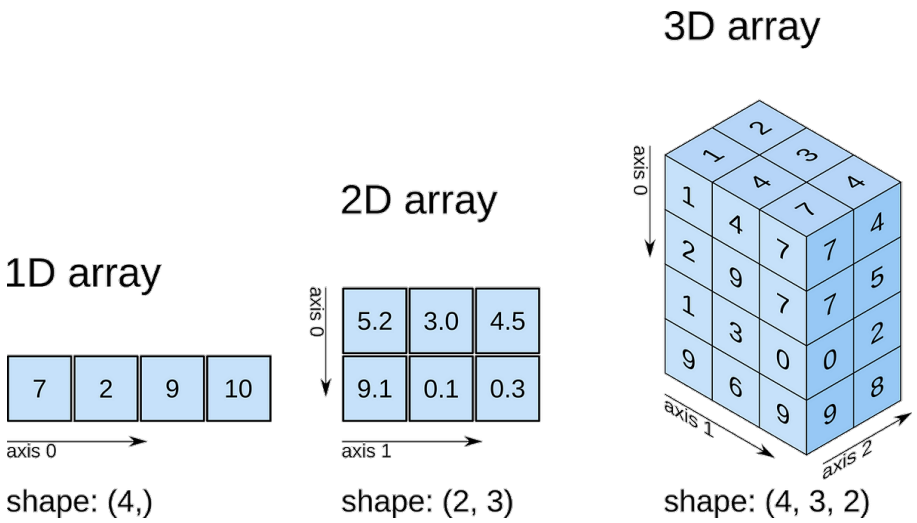
- **What is NumPy?:** NumPy (Numerical Python) is a fundamental package for scientific computing in Python. It provides support for arrays, matrices, and many mathematical functions.
- **What is Pandas?:** Pandas is a powerful data manipulation and analysis library for Python. It offers data structures like DataFrame and Series for efficient data handling.
- **Importance:** Both libraries are essential for data science, enabling efficient computation, data manipulation, and analysis.

NumPy Basics

- **Introduction to NumPy:** NumPy (Numerical Python) is a powerful library for numerical computations. It provides support for arrays, matrices, and a wide range of mathematical functions.
- **Installation:** Install NumPy using pip: `pip install numpy`. It's often included in scientific Python distributions like Anaconda.
- **Setup:** To start using NumPy, import it in your Python script: `import numpy as np`.

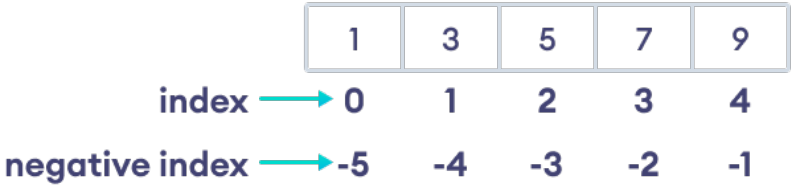
NumPy Arrays

- **Creating Arrays:** NumPy arrays can be created using functions like `np.array()`, `np.zeros()`, `np.ones()`, and `np.arange()`.
- **Array Attributes:** Arrays have attributes such as shape, size, and dtype. For example, `array.shape` gives the dimensions of the array.
- **Array Operations:** NumPy supports element-wise operations, matrix multiplication, and many mathematical functions that can be applied to arrays.



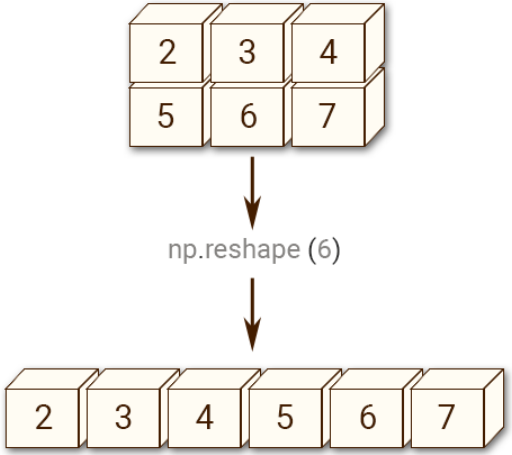
NumPy Array Indexing

- **Basic Indexing:** Access array elements using indices, e.g., `array[0]` for the first element. Use `array[-1]` for the last element.
- **Slicing:** Extract subarrays using slices, e.g., `array[1:5]` for elements from index 1 to 4. Use `:` to denote the full range.
- **Advanced Indexing:** NumPy supports advanced indexing techniques such as boolean indexing and fancy indexing with lists or arrays of indices.



NumPy Array Manipulation

- **Reshaping Arrays:** Change the shape of an array using `np.reshape()`. For example, convert a 1D array to a 2D array.
- **Concatenation:** Combine arrays using functions like `np.concatenate()`, `np.vstack()`, and `np.hstack()`.
- **Splitting Arrays:** Split arrays into multiple subarrays using `np.split()`, `np.hsplit()`, and `np.vsplit()`.



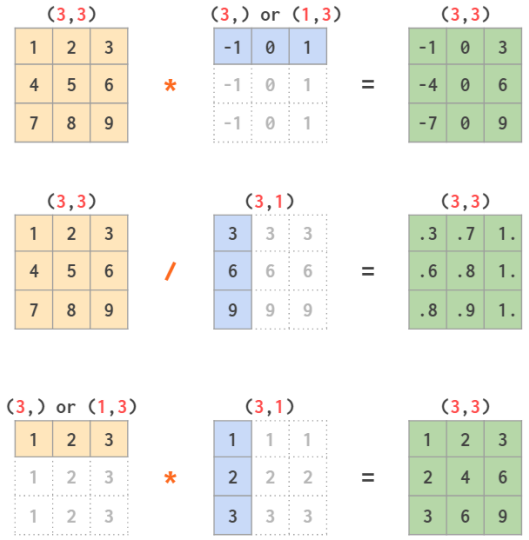
NumPy Mathematical Functions

- **Basic Mathematical Operations:** NumPy supports basic operations such as addition, subtraction, multiplication, and division. For example, `np.add()` and `np.subtract()`.
- **Statistical Functions:** Calculate statistics like mean, median, and standard deviation using functions like `np.mean()`, `np.median()`, and `np.std()`.
- **Linear Algebra:** Perform linear algebra operations like matrix multiplication, eigenvalue decomposition, and singular value decomposition with `np.linalg` module.

Operator	ufunc	Description
+	np.add	Addition
-	np.subtract	Subtraction
-	np.negative	Unary negation
*	np.multiply	Multiplication
/	np.divide	Division
//	np.floor_divide	Floor division
**	np.power	Exponential
%	np.mod	Modulus or remainder

NumPy Broadcasting

- **Concept of Broadcasting:** Broadcasting allows NumPy to perform operations on arrays of different shapes, automatically expanding the smaller array.
- **Examples of Broadcasting:** Common use cases include adding a vector to a matrix and multiplying arrays of different dimensions.
- **Benefits:** Broadcasting simplifies code and improves performance by avoiding explicit loops.



multiplying several columns at once

row-wise normalization

outer product

NumPy Universal Functions (ufuncs)

- **Introduction to ufuncs:** Universal functions (ufuncs) are functions that operate element-wise on arrays. Examples include `np.add()`, `np.sin()`, and `np.exp()`.
- **Benefits of ufuncs:** Ufuncs are optimized for performance, often implemented in C, and can handle broadcasting.
- **Examples of ufuncs:** Examples include trigonometric functions like `np.sin()`, exponential functions like `np.exp()`, and aggregation functions like `np.sum()`.

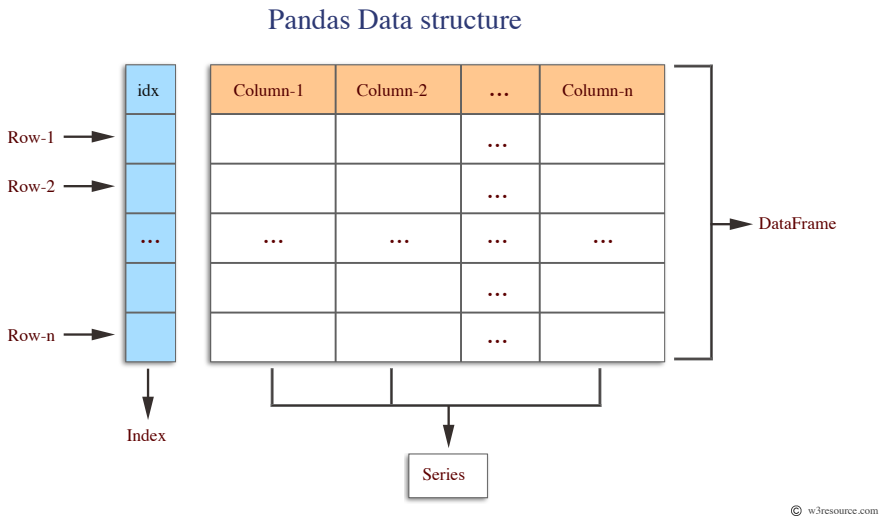
Pandas Basics

- **Introduction to Pandas:** Pandas is a powerful data manipulation and analysis library for Python, providing data structures like Series and DataFrame.
- **Installation:** Install Pandas using pip: `pip install pandas`. It is also included in scientific Python distributions like Anaconda.
- **Setup:** To start using Pandas, import it in your Python script: `import pandas as pd`.

Pandas

Pandas Data Structures

- **Series:** A Pandas Series is a one-dimensional labeled array capable of holding data of any type (integer, string, float, etc.). It is similar to a column in an Excel spreadsheet.
- **DataFrame:** A Pandas DataFrame is a two-dimensional labeled data structure with columns of potentially different types. It is similar to a table in a relational database or an Excel spreadsheet.
- **Creating DataFrames:** DataFrames can be created from various data structures, including lists, dictionaries, and NumPy arrays.

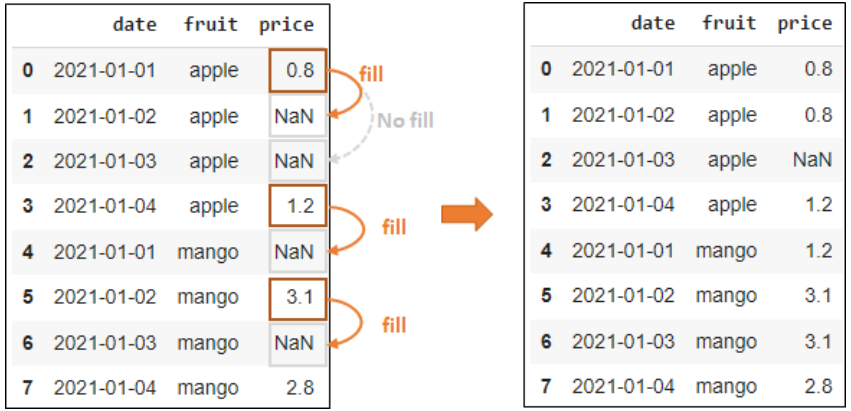


Pandas Data Indexing and Selection

- **Indexing:** Pandas provides several ways to index data, including label-based indexing with `loc` and position-based indexing with `iloc`.
- **Selecting Data:** Select data using methods like `[]`, `.loc[]`, `.iloc[]`, and `.at[]`. Boolean indexing allows filtering data based on conditions.
- **Examples:** Use `.loc[]` for label-based selection: `df.loc['row_label']`. Use `.iloc[]` for position-based selection: `df.iloc[0]`.

Pandas Handling Missing Data

- **Identifying Missing Data:** Use functions like `isnull()` and `notnull()` to identify missing data in a DataFrame. The `sum()` method can be used to count missing values.
- **Handling Missing Data:** Handle missing data by dropping or filling them. Use `dropna()` to remove missing values and `fillna()` to replace them with a specified value or method.
- **Examples:** Drop rows with missing values: `df.dropna()`. Fill missing values with the mean: `df.fillna(df.mean())`.



	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	NaN
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	NaN
7	2021-01-04	mango	2.8

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	0.8
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	1.2
5	2021-01-02	mango	3.1
6	2021-01-03	mango	3.1
7	2021-01-04	mango	2.8

Pandas Data Merging and Joining

- **Merging DataFrames:** Merge DataFrames using the `merge()` function, similar to SQL joins. Specify the columns to join on using the `on` parameter.
- **Joining DataFrames:** Use the `join()` method to join DataFrames based on their indices. This is similar to merging but uses the index by default.
- **Examples:** Merge two DataFrames on a common column: `pd.merge(df1, df2, on='key')`. Join DataFrames on their index: `df1.join(df2)`.

df1:

	A_col1	A_col2	A_col3
0	1	1	w
1	2	1	x
2	3	2	y
3	4	3	z

,

df2:

	B_col1	B_col2	B_col4
0	1	1	10
1	2	2	11
2	3	2	12

```
df1.merge(df2, left_on=['A_col1', 'A_col2'], right_on=['B_col1', 'B_col
```

	A_col1	A_col2	A_col3	B_col1	B_col2	B_col4
0	1	1	w	1	1	10
1	3	2	y	3	2	12

```
df1.merge(df2, left_on=['A_col1', 'A_col2'], right_on=['B_col2', 'B_col
```

	A_col1	A_col2	A_col3	B_col1	B_col2	B_col4
0	1	1	w	1	1	10

Photo by Sid Balachandran on Unsplash

Pandas Grouping and Aggregation

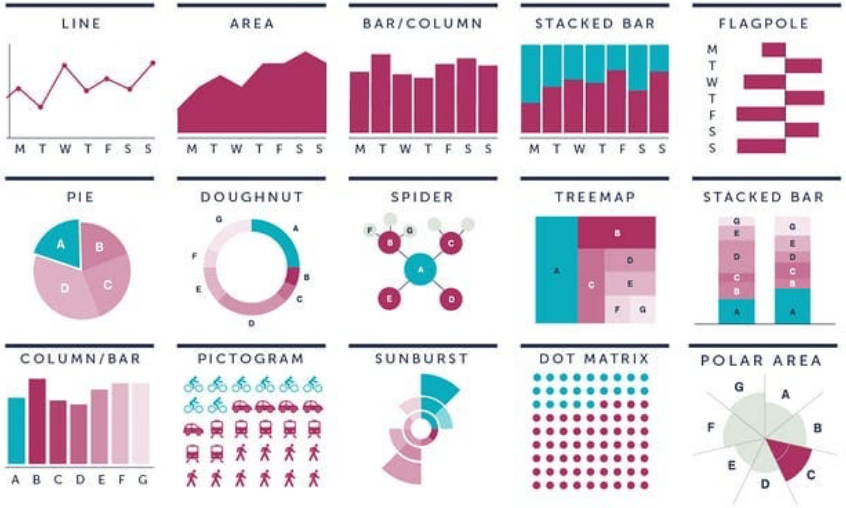
- **GroupBy Operations:** Group data using the `groupby()` method, which allows you to split data into groups based on some criteria.
- **Aggregation Functions:** Apply aggregation functions to grouped data, such as `sum()`, `mean()`, and `count()`.
- **Examples:** Group data by a column and calculate the mean: `df.groupby('column').mean()`. Aggregate multiple functions: `df.groupby('column').agg(['sum', 'mean'])`.

In [10]:
```
multi_grouped = df.groupby(['A', 'C'])

for name, group in multi_grouped:
    print("Name:", name)
    print("Group:")
    print(group, "\n")
```

```
Name: (1, 5)
Group:
   A   B  C
0  1  10  5

Name: (1, 15)
Group:
   A   B   C
1  1  20  15
```

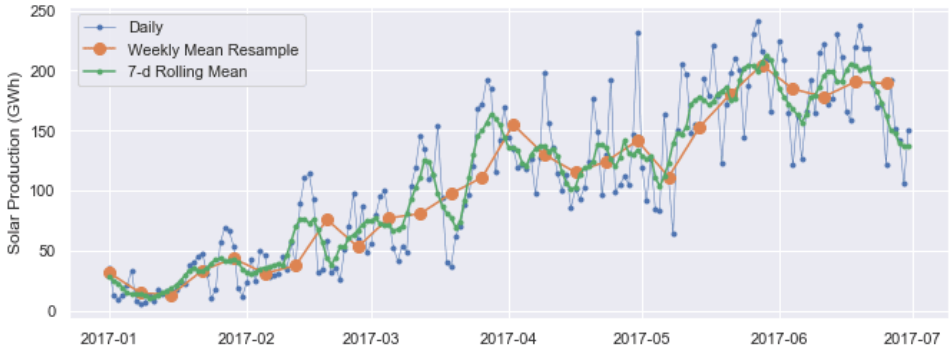
Pandas Data Visualization

- **Plotting Functions:** Pandas provides built-in functions for plotting data directly from DataFrames and Series. Use methods like `plot()`, `hist()`, and `boxplot()`.
- **Integration with Matplotlib:** Pandas' plotting capabilities are built on Matplotlib, allowing for extensive customization and integration with Matplotlib functions.
- **Examples:** Create a line plot: `df.plot()`. Create a histogram: `df.hist()`. Create a boxplot: `df.boxplot()`.



Pandas Data Input and Output

- **Reading Data:** Read data from various file formats such as CSV, Excel, and SQL databases using functions like `read_csv()`, `read_excel()`, and `read_sql()`.
- **Writing Data:** Write data to files using functions like `to_csv()`, `to_excel()`, and `to_sql()`.
- **Examples:** Read a CSV file: `df = pd.read_csv('file.csv')`. Write to an Excel file: `df.to_excel('file.xlsx')`.

Pandas Time Series Analysis

- **Time Series Data:** Pandas provides robust support for working with time series data, including date indexing, resampling, and time zone handling.
- **Date Indexing:** Set a column as the date index to enable time series operations. Use functions like `pd.to_datetime()` to convert dates.
- **Examples:** Resample data to monthly frequency: `df.resample('M').mean()`. Shift data forward or backward: `df.shift(1)`.